Avaliação de Desempenho em Pipelines ETL Python: Uma Abordagem Comparativa entre Pandas e Polars

Matheus Duarte da Silva, *Estudante, UnB,*

***Abstract*—This paper presents a data processing performance comparison between the Pandas and Polars libraries in Python. For this analysis, a synthetic dataset of 4.67 GB containing 73 million records was used, and execution times for classic ETL (Extract, Transform, Load) operations were measured. The results indicate that the multithreaded architecture and Lazy Evaluation approach of Polars reduced the total processing time from approximately 172 seconds in Pandas to about 4.2 seconds. This represents a significant execution time reduction of 97.5%, demonstrating that Polars was approximately 41 times faster than Pandas in this scenario.**

***Index Terms*—Python, Polars, Pandas, ETL, Multithread, Lazy Evaluation, Eager, dataset**

***Resumo*—Este artigo apresenta uma comparação de desempenho no processamento de dados entre as bibliotecas Pandas e Polars em Python. Para a análise, utilizou-se um dataset sintético de 4,67 GB contendo 73 milhões de registros, sobre o qual foram realizadas medições do tempo de execução em operações clássicas de ETL (Extração, Transformação e Carga). Os resultados apontam que a arquitetura multithread e a avaliação preguiçosa (Lazy Evaluation) do Polars reduziram o tempo total de processamento de aproximadamente 172 segundos, no Pandas, para cerca de 4,2 segundos. Isso representa uma redução significativa de 97,5% no tempo de execução, indicando que o Polars demonstrou ser aproximadamente 41 vezes mais rápido que o Pandas neste cenário.**

***Palaras-chave*—Python, Polars, Pandas, ETL, Multithread, Lazy Evaluation, Eager, dataset**

I. Introdução

COM o crescimento explosivo do volume de dados gerados diariamente [1], devido ao alto nível de conectividade global, a necessidade de armazenar, processar e analisar informações tornou-se o foco central das empresas e organizações [2]. Relatórios da indústria estimam que o volume global de dados gerados alcançou 147 zettabytes em 2024, com projeções indicando um crescimento contínuo para 181 zettabytes em 2025 e 221 zettabytes até 2026 [3].

Diante desse cenário de Big Data, o processamento eficiente dessas informações tornou-se um desafio computacional de alta importância. A linguagem Python consolidou-se como o padrão principal para essas tarefas, liderando os índices de popularidade no ecossistema de análise de dados [4]. Historicamente, a biblioteca Pandas tem sido a escolha tradicional da comunidade para operações de Extração, Transformação e Carga (ETL) de dados.

No entanto, à medida que os datasets ultrapassam a escala dos gigabytes, o Pandas apresenta limitações severas de desempenho. [5] Por utilizar uma arquitetura single-thread (execução em núcleo único) e realizar avaliações de processamento de forma imediata (Eager Evaluation), a biblioteca frequentemente acaba com a memória RAM disponível e gera alto tempo de espera para operações complexas e com grandes volumes de dados [6]. Como alternativa a essa lacuna, o Polars, uma biblioteca recente desenvolvida principalmente na linguagem Rust, tem ganhado destaque ao introduzir o paralelismo (multi-threading) e a otimização de consultas por avaliação preguiçosa (Lazy Evaluation) em ambientes de processamento local [7].

Assim, este artigo tem como objetivo comparar o desempenho computacional entre as bibliotecas Pandas e Polars em cenários de processamento de dados em larga escala. Ao analisar o tempo de execução de operações clássicas de manipulação em um conjunto de dados de 4,67 GB, o estudo visa destacar os impactos arquiteturais de cada ferramenta e fornecer informações técnicas para que seja possível escolher a melhor opção para lidar com grandes volumes de dados em ambientes locais.

MDS
24 de Abril de 2026

II. Metodologia

A. Especificações da Máquina Utilizada

Para garantir a reprodutibilidade e fornecer o contexto para a análise de desempenho, o experimento foi conduzido em um máquina com um processador de 13ª Geração Intel Core i5-13420H, contendo 12 núcleos físicos, 16 GB de memória RAM e sistema operacional Fedora Linux 43. A disponibilidade de múltiplos núcleos de processamento é um fator arquitetural muito importante para a metodologia deste estudo, pois é com ele que podemos verificar a questão do paralelismo (*multi-threading*) do Polars em contraste direto com a arquitetura de execução em um só núcleo (*single-thread*) do Pandas.

B. Conjunto de Dados (Dataset)

O *benchmark* utilizou um conjunto de dados sintético (*dataset*) no formato CSV gerado dentro do arquivo notebook, que contém o experimento, por meio de um script Python, que tem 73 milhões de registros e por fim ocupou 4,67 GB de armazenamento na máquina. Esse dataset está disponível no repositório deste projeto [8]. Este volume de dados foi escolhido com intuito de representar uma carga de trabalho crítica: ao ser processado sob avaliação imediata (*Eager Evaluation*),

o *dataset* aproxima-se do limite de memória RAM disponível no sistema (16 GB), chegando na marca de 15 GB utilizado durante a execução do programa, assim mostrando os gargalos clássicos de gerenciamento de memória no Pandas. A estrutura dos dados deste dataset está detalhada na Tabela I.

TABLE I
DICIONÁRIO DE DADOS DO DATASET SINTÉTICO

Nome da Coluna	Tipo de Dado
id	`integer`
name	`string`
date	`datetime`
sensor_status	`string`
category	`string`
latitude	`float`
longitude	`float`

Para simular o comportamento de *pipelines* de dados do mundo real, a coluna `sensor_status` foi populada, de forma randômica, com alguns casos de falhas e valores nulos (`NaN`) em seus campos. Essa técnica de introdução de ruído garante que as bibliotecas sejam submetidas ao processamento de dados ausentes, uma etapa custosa, porém de grande importância em operações modernas de Engenharia de Dados.

C. Operações ETL Avaliadas

O experimento consistiu na execução sequencial de sete operações fundamentais em operações de Extração, Transformação e Carga (ETL). O tempo de execução de cada etapa foi medido através de funções nativas de medição de tempo do próprio Python (`time`). As operações foram desenhadas para estressar tanto as operações de Entrada/Saída (I/O) quanto a CPU em si:

- **Carga de Dados (I/O):** Leitura e alocação do arquivo CSV na memória
- **Mapeamento de Nulos:** Varredura do *dataframe* para contabilização de dados faltantes
- **Limpeza (*Drop Nulls*):** Filtragem e remoção dos itens que estavam nulos
- **Engenharia de *Features*:** Criação de uma coluna que precisa calcular a soma entre duas colunas (`latitude` e `longitude`) e realizar operação de divisão com números em ponto flutuante
- **Filtragem Lógica:** Filtro de registros com base em operações booleanas
- **Agrupamento e Agregação:** Utilização de operações *Group By* com funções de agregamento
- **Ordenação de Dados (*Sorting*):** Organização decrescente de todo o *dataset* baseada na coluna de data, é uma das operações de maior complexidade algorítmica em estruturas não indexadas

III. RESULTADOS E DISCUSSÃO

A avaliação de desempenho foi dividida em duas etapas principais. Primeiramente, foi comparada a execução imediata (*Eager Evaluation*) de ambas as bibliotecas, operação por operação. Em seguida, avaliou o impacto da otimização de consultas nativa do Polars através da avaliação preguiçosa (*Lazy Evaluation*).

A. Desempenho por Operação (Eager Evaluation)

Na abordagem de execução imediata, onde cada operação no *dataframe* é alocado e realizado na memória antes da execução da próxima, os resultados evidenciaram uma disparidade nas arquiteturas entre as ferramentas, conforme ilustrado na Fig. 1 abaixo.

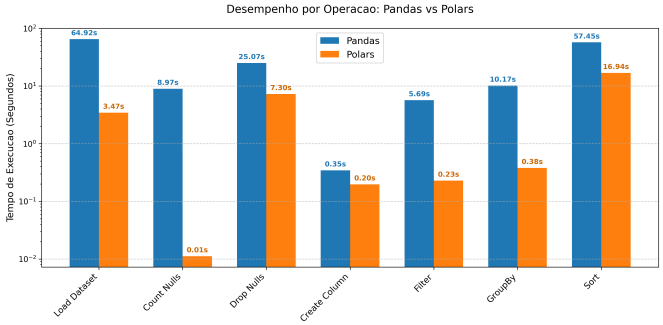

Fig. 1. Desempenho por Operações

O Pandas apresentou gargalos significativos em operações que exigem alta alocação de memória e leitura total do conjunto de dados. A etapa de carga do *dataset* (I/O) consumiu aproximadamente 61,2 segundos no Pandas, contra apenas 3,3 segundos no Polars. Essa diferença é justificada pela implementação de leitura de arquivos em Rust do Polars, que distribui o *parsing* do CSV em múltiplas *threads*, que fornece uma paralelização das tarefas.

A discrepância mais perceptivél ocorreu na de ordenação dos dados (*Sorting*). Devido à sua natureza *single-thread*, o Pandas levou cerca de 56,2 segundos para ordenar os 73 milhões de registros, enquanto que o paralelismo do Polars concluiu a mesma tarefa em 13,3 segundos. Mesmo em operações aritméticas simples de criação de colunas, o Polars manteve a superioridade computacional, demonstrando maior eficiência no acesso aos dados em memória estendida.

B. Otimização de Consultas e Lazy Evaluation

Embora o Polars já demonstre superioridade no modo *Eager*, seu principal diferencial para cenários de Engenharia de Dados em larga escala (Big Data) reside na *Lazy Evaluation*. A Fig. 2 apresenta o tempo total de processamento necessário para finalizar todo o *pipeline* ETL testado.

A execução sequencial de todas as operações no Pandas totalizou cerca de 172 segundos. Quando o mesmo fluxo foi construído utilizando a API *Lazy* do Polars (através do método `scan_csv` seguido de `collect()`), o tempo total de processamento despencou para notáveis 4,7 segundos. Mostrando uma melhoria do tempo de processo absurda.

Esse expressivo ganho de desempenho acontece porque, ao utilizar a API *Lazy*, o Polars não executa o código linha por linha de forma imediata [9]. Em vez disso, a biblioteca lê todas as intenções do desenvolvedor e constrói um plano de consulta otimizado (*Query Plan*) antes mesmo de acessar o arquivo na memória [10].

Durante essa otimização, o motor do Polars descobre de antemão quais linhas devem ser filtradas (*Predicate Pushdown*)

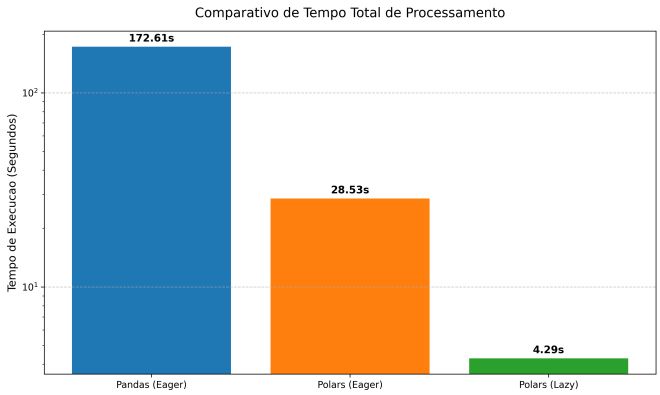

Fig. 2. Comparativo de Tempo Toal

e quais colunas serão realmente utilizadas (*Projection Push-down*) [10], [11]. Assim, ele extrai e carrega para a memória RAM apenas os fragmentos de dados estritamente necessários. Como resultado, o sistema não desperdiça recursos criando tabelas intermediárias, o que reduz drasticamente a sobrecarga e torna o Polars a ferramenta ideal para processar grandes volumes de dados em máquinas com *hardware* limitado.

IV. Conclusão

Este artigo apresentou uma análise comparativa de desempenho entre as bibliotecas Pandas e Polars em rotinas de processamento de dados massivos. Os experimentos demonstraram que, ao lidar com um *dataset* de 4,67 GB, as limitações da arquitetura *single-thread* e da avaliação imediata do Pandas resultam em severos gargalos computacionais, esgotando rapidamente os recursos de memória e CPU.

Em contrapartida, o Polars provou ser uma alternativa estruturalmente superior para este cenário, reduzindo o tempo total de execução do *pipeline* em 97,5%, essa melhora mostra que a biblioteca opera aproximadamente 41 vezes mais rápida. Essa eficiência é justificada pela sua implementação em Rust, que viabiliza o processamento paralelo em múltiplos núcleos, e pela aplicação da *Lazy Evaluation*, que otimiza o plano de consulta antes da alocação de memória.

Assim, podemos concluir que para engenheiros de dados e organizações que processam volumes de informações na escala de *gigabytes* ou até mesmo em escalas acima do *gigabytes*, em infraestruturas locais, a adoção do Polars representa um ganho técnico e operacional interessante.

Para poder compartilhar de maneira clara e de fácil reprodução deste experimento, todos os *scripts* de geração de dados e *notebooks* de *benchmark* foram disponibilizados publicamente no repositório Benchmark ETL Python: Pandas vs Polars [8].

References

[1] F. Duarte, "Amount of data created daily (2026)," Feb 2026. [Online]. Available: https://explodingtopics.com/blog/data-generated-per-day%26nbsp

[2] R. Zem, "Os 25 empregos que mais devem crescer em 2026," Jan 2026. [Online]. Available: https://g1.globo.com/trabalho-e-carreira/noticia/2026/01/07/empregos-alta-ranking-linkedin.ghtml

[3] N. Kumar, "Big data statistics 2026 (growth, trends market size)," Dec 2025. [Online]. Available: https://www.demandsage.com/big-data-statistics/

[4] A. Di Battista, S. Grayling, X. Játiva, T. Leopold, R. Li, S. Sharma, and S. Zahidi, "Future of jobs report 2025," Jan 2025. [Online]. Available: https://reports.weforum.org/docs/WEF_Future_of_Jobs_Report_2025.pdf

[5] A. Spark, "Pandas api on spark," 2024. [Online]. Available: https://spark.apache.org/pandas-on-spark/

[6] Databricks. (2024) Databricks — polars vs pandas. [Online]. Available: https://www.databricks.com/blog/polars-vs-pandas

[7] P. Staff, "Blazingly fast dataframe library," Oct 2025. [Online]. Available: https://docs.pola.rs/#key-features

[8] M. D. da Silva. (2026, Apr) Benchmark etl python: Pandas vs polars. Repositório no GitHub contendo os códigos e dados do experimento. [Online]. Available: https://github.com/smmstakes/benchmark-etl-pandas-x-polars

[9] Polars Contributors. (2024) Usage - polars user guide: The lazy api. [Online]. Available: https://docs.pola.rs/user-guide/lazy/using/

[10] ——. (2024) polars_lazy - rust api reference. [Online]. Available: https://docs.pola.rs/api/rust/dev/polars_lazy/index.html

[11] ——. (2024) Cloud storage - polars user guide. [Online]. Available: https://docs.pola.rs/user-guide/io/cloud-storage/